

## Projet 2 - Exploration des modèles de langue

Suite à la phase d'annotation et d'adjudication, vous devez entraîner un modèle pour annoter automatiquement des textes médicaux avec les types de concept d'UMLS.

Consignes générales

- Le code livré pour chaque étape (>0) doit partir du code de l'étape précédente et inclure les modifications minimales pour atteindre l'objectif demandé.  
**Ne pas modifier ou remplacer la majorité du code par une autre structure.**
- Une seule remise sera effectuée pour l'ensemble du projet.  
**Assurez-vous de tout inclure les livrables selon la structure exigée.**
- Les fichiers de code, scripts et dépendances **doivent être installables et exécutables sur le serveur du cours.**

Si l'une ou plusieurs de ces consignes ne sont pas respectées, des pénalités seront encourues.

### Étape 0 - Entraîner un modèle de test sur Slurm

**Objectif d'apprentissage:** être capable d'entraîner un modèle sur une grappe de calcul

Pour vous familiariser avec l'entraînement de modèles sur une grappe de calcul, du code de base vous est fourni pour aligner un modèle sur une tâche de reconnaissance d'entité nommée avec le corpus CONLL2003.

Utilisez le fichier `train_conll2003.py` dans l'archive de code sous la section projet 2 du cours. Il est basé sur un exemple tiré de la librairie de HuggingFace. Lisez bien le code de ce fichier pour comprendre comment le chargement des données, l'entraînement et l'évaluation fonctionnent, car ils constituent la base de travail pour le projet.

Débutez par vous créer un environnement virtuel sur le serveur et installer les librairies nécessaires. Par la suite, tel qu'indiqué dans la vidéo de fonctionnement de Slurm sur le site du cours, vous pouvez créer un script d'exécution pour soumettre la tâche d'entraînement au serveur.

Voici les paramètres à utiliser dans votre script d'exécution sur Slurm pour lancer l'entraînement:

```
--model_name_or_path bert-base-uncased
--dataset_name conll2003
--output_dir test-ner
--do_train
--do_eval
--trust_remote_code=True
```

Assurez-vous de gérer les paramètres pour que les modèles se créent dans un sous-répertoire du répertoire de votre tâche.

**Livable:** Aucun, mais assurez-vous de faire cette étape en premier.

## **Étape 1 - Modifier le code pour s'entraîner sur MedMention**

**Objectif d'apprentissage:** être capable d'entraîner un modèle sur un jeu de données publiques.

Maintenant que vous pouvez entraîner un modèle avec succès sur le serveur à l'aide de slurm, l'étape 1 consiste à

Vous devez faire les modifications suivantes:

- S'assurer que les étiquettes sont bien spécifiées dans le modèle
- Assurez-vous de sauvegarder les différentes versions du modèle au cours de l'entraînement pour que le processus soit interruptible
- Sauvegarder les checkpoints dans un sous-répertoire à partir de votre script d'entraînement
- Limiter le nombre de checkpoint du modèle à un maximum de 3
- Chargé le meilleure modèle avant l'évaluation
- Entraîner minimalement pendant 5 epoch
- Faites quelques tests manuels pour modifier les hyperparamètres d'entraînement pour tenter d'obtenir un modèle plus performant.

### Jeu de données MedMention

Vous utiliserez les données annotées de [MedMention](#) qui contiennent des titres d'articles récupérés sur PubMed et autres bases de données publiques.

Ce jeu de données est disponible en chargement direct dans la librairie Datasets sous le nom "ibm-research/MedMentions-ZS", préformaté en format [IOB2](#) et utilisant comme étiquette les types sémantiques de chaque concept. Voici un exemple:

*[ "DCTN4", "as", "a", "modifier", "of", "chronic",  
"Pseudomonas", "aeruginosa", "infection", "in",  
"cystic", "fibrosis" ]*

*[ "B-T103", "O", "O", "O", "O",  
"B-T038", "I-T038", "I-T038",  
"I-T038", "O", "B-T038", "I-T038" ]*

La colonne de droite contient le titre séparé en mots. Celle de droite contient l'étiquette correspondante à chaque mot avec le code du type sémantique tiré d'UMLS.

Ce jeu de données contient 26k titres pour l'entraînement, 1,2k pour la validation et un peu plus de 1k pour l'évaluation (test).

Effectuer une analyse des données et des résultats ((voir la méthode à la fin de cet énoncé)) de ce modèle. Ceci constitue votre modèle de référence (ou *baseline* en anglais) de comparaison.

**Livrables:**

- Une analyse des données et résultats de votre meilleur modèle à inclure dans le rapport final sous la mention “Étape 1”. Décrivez les hyperparamètres testés et ceux qui ont obtenu les meilleures performances.
- Sous un répertoire nommé “etape1”
  - Code modifié pour réaliser cette étape avec les lignes modifiées identifiées
  - Le script .sh utilisé pour exécuter la tâche sur slurm
  - Les fichiers complets d’erreurs et de sorties
  - Un fichier contenant les prédictions du modèle sur le jeu d’évaluation (format iob)

## **Étape 2 - Ajouter des données annotées du cours MTI881**

**Objectif d’apprentissage:** augmentation et gestion des données

Vous avez accès par le site du cours à l’ensemble des données annotées par les différentes équipes dans le projet 1.

Votre tâche est d’intégrer le maximum de ces données dans l’entraînement et l’évaluation de votre modèle. Pour chaque équipe, indiquer quelles données ont été utilisées, et si aucune donnée n’a été retenue pour une équipe, expliquer la raison (données invalides, etc).

Vous devez prendre en compte la distribution des données existantes et celles qui s’y ajouteront. Vous devez vous assurer que les classes à prédire sont aussi balancées qu’il est possible avec les données disponibles, et ce pour chaque segment (entraînement, validation, évaluation).

**Vous pouvez convertir facilement les “concept id” annotés vers des types sémantiques** avec le script fourni dans le répertoire umls dans le code source initial.

Pour vous faciliter la vie, vous pouvez vous créer un nouveau jeu de données sur la même structure que MedMention et utiliser la fonction “*concatenate\_datasets*” de la librairie dataset. Assurez-vous de combiner les fragments correspondants.

**Livrables:**

- Une analyse des données et résultats (voir la méthode à la fin de cet énoncé) à inclure dans le rapport final sous la mention “Étape 2”
- Sous un répertoire nommé “etape2”
  - Code modifié pour réaliser cette étape avec les lignes modifiées identifiées

- Les nouvelles données dans le format nécessaire pour le chargement lors de l'entraînement avec votre code source
- Le script .sh utilisé pour exécuter la tâche sur slurm
- Les fichiers complets d'erreurs et de sorties
- Un fichier contenant les prédictions du modèle sur le jeu d'évaluation (format iob)

### **Étape 3 - Modifier l'entraînement pour utiliser un modèle génératif**

**Objectif:** utilisation comparative d'un modèle génératif

Pour cette étape, changer le modèle utilisé au modèle GPT Neo 1.3B d'EleutherAI. Vous devrez peut-être quelques éléments de votre script pour que le modèle se charge correctement en mémoire et puisse être entraîné, selon vos modifications de la phase précédente.

**Livrables:**

- Une analyse des données et résultats (voir la méthode à la fin de cet énoncé) à inclure dans le rapport final sous la mention "Étape 3"
- Sous un répertoire nommé "etape3"
  - Code modifié pour réaliser cette étape avec les lignes modifiées identifiées
  - Le script .sh utilisé pour exécuter la tâche sur slurm
  - Les fichiers complets d'erreurs et de sorties
  - Un fichier contenant les prédictions du modèle sur le jeu d'évaluation (format iob)

### **Étape 4 - Thème libre**

**Objectif d'apprentissage:** explorer les améliorations possibles en lien avec vos observations précédentes.

À partir d'un des modèles précédents, idéalement le plus performant, plusieurs options s'offrent à vous:

- Augmenter significativement les données d'entraînement:
  - volume
  - qualité
  - bruitez les données à l'entraînement
  - Stratification
- Intégrer un processus d'optimisation d'hyperparamètres (avec sherpa ou autre) et de modification des composants d'entraînement pour améliorer le modèle résultant
  - Fonction de coût
  - Optimizer (peut demander de quantizer le modèle dans certains cas)
  - Régime d'entraînement
  - etc

- Changer la structure d'entraînement (adapter, distillation, multitâches, etc)
- Ajouter la prédiction de relations avec les données annotées du premier projet (ensemble des projets des différentes équipes)

Le jeu d'évaluation est fixe et les données ne doivent jamais être contenues dans le jeu d'entraînement.

L'objectif est de comparer les différentes approches et de qualifier leur impact sur le jeu d'évaluation.

#### **Livrables:**

- Une description du choix de l'option basé sur les analyses et résultats des modèles précédents. Expliquer votre choix et votre approche dans le rapport final sous la mention "Étape 4".
- Une analyse des données et résultats (voir la méthode à la fin de cet énoncé) à inclure dans le rapport final.
- Sous un répertoire nommé "etape4"
  - Code modifié pour réaliser cette étape avec les lignes modifiées identifiées
  - Le script .sh utilisé pour exécuter la tâche sur slurm
  - Les fichiers complets d'erreurs et de sorties
  - Un fichier contenant les prédictions du modèle sur le jeu d'évaluation (format iob)

### **Réaliser une analyse des données et résultats**

Pour chaque étape, sauf l'étape 0, vous devez analyser les données entrant dans le processus d'entraînement (entraînement, validation, évaluation) ainsi que les résultats que le modèle vous fournit sur les données d'évaluation.

Assurez-vous de placer le code d'évaluation dans un fichier séparé du code d'entraînement.

Étudier le contenu de données sources, notamment les éléments suivants:

- Nombre de classes
- Cooccurrence des classes par instance (les titres d'articles)
- Décrivez les tendances morphologique et syntaxique des formes de surface par classe

Pour les résultats, calculez et commentez sur les points suivants:

- Quels sont les scores de performance globaux et par classe?
  - Est-ce qu'il y a une grande variance? Pourquoi?
- Dénombrer les différents types d'erreurs (surgénérations, annotations manquantes, mauvais type, mauvaises frontières, etc) par classe.
  - Quelles sont les tendances?

- Quelles sont les sources probables de ces erreurs en lien avec l'architecture et les données utilisées?
- Dénombrer les tendances dans les erreurs pour chaque classe en lien avec les éléments linguistiques du premier cours.
  - Dans quels contextes (parti du discours, types de syntagme, morphologie, etc) ces erreurs surviennent-elles?
  - Donner des exemples concrets d'erreurs pour chaque contexte.
  - Est-ce que ces contextes sont présents dans les données d'entraînement et de validation? Pour la même classe ou d'autres?

Utiliser des figures et exemples lorsque c'est pertinent pour aider la lecture, mais n'oubliez pas de fournir les nombres demandés, par exemple dans un tableau récapitulatif par section.

Il est recommandé de préparer le code d'évaluation dès la première phase car elle sera nécessaire pour l'ensemble des phases suivantes.

### **Rapport technique**

Remettre un rapport en format PDF détaillant l'ensemble des analyses de résultats et d'erreurs pour les étapes de 1 à 4 inclusivement.

De plus, pour l'étape 4, décrire le thème choisi et motiver la raison de ce choix basé sur les résultats des étapes précédentes, Détailler le processus suivi et mettre en référant aux fichiers (code ou données) remis.

Rédiger une discussion sur les constats que vous avez fait sur les résultats et erreurs de chaque type de modèle. Décrivez, dans un contexte de recherche d'informations pour le domaine médical, quels sont les impacts potentiels des erreurs de votre meilleur modèle sur cette tâche.

Allez à l'essentiel de ce qui est demandé: c'est un rapport technique et non un roman fleuve. ;)

### **Structure de la remise du projet**

La remise du projet doit être effectuée sur le site Moodle du cours avec un seul fichier d'archive (zip) avec la structure indiquée ci-dessous. (Changer X pour votre numéro d'équipe)

- RapportEquipeX.pdf
- etape1
  - code\_source\_python.py
  - tache\_slurm.sh
  - (autres fichiers nécessaires, erreur, sorties, prédiction, etc)
- etape2

- code\_source\_python.py
- tache\_slurm.sh
- fichier\_données\_supplémentaires.csv (ou autre format)
- (autres fichiers nécessaires, ...)
- etape3
  - code\_source\_python.py
  - tache\_slurm.sh
  - (autres fichiers nécessaires, ...)
- etape4
  - code\_source\_python.py
  - tache\_slurm.sh
  - (autres fichiers nécessaires, ...)